2026-11-01 07:57:56 You

1. Automated infrastructure provisioning and configuration management using ansible playbook to accelrate deployment.
2. Developed reusable ansible playbook and roles to automate provisioning and configuration and patching of kubernetes cluster across multiple env.
3. Built end to end jenkins pipeline that automate frontend application compliation, containise image and trigger seamless deployment to cluster
4. Created and managed helm charts and kubernetes manifest to standardise env configuration and simplify application lifecycles
5. Configured cluster resource managment by defining CPU/Memory request and limits, resourceQuotas, limitranges to optimize pod scheduling and prevent resource starvation
6. Implemented application autoscaling by configuring HPA based on CPU and memory utilization metric to handle traffic spikes smoothly
7. Implemented rolling updates and rollback strategies using kubernetes deployment configuration to ensure zero downtime application upgrades and quick recovery from failures
8. Optimized container networking by trobleshooting cluster internal DNS issues, service routing and manging ingress/Route controllers for external traffic distribution

For every points

1\. come up with the simple and clear technical definition

2. Good story telling way to memorise things
3. Task with an incident that might happen in my previous project
4. Erros/Issues i might have faced on my previous project

ChatGPT gpt-6

DevOps Project – Client Interview Preparation

We'll treat these eight responsibilities as parts of one consistent enterprise application rather than eight unrelated technologies.

Project scenario: An enterprise web application serving lakhs of users across India, deployed on AWS EKS. It has a frontend application, backend services and supporting infrastructure. We use Jenkins for CI/CD, Ansible for configuration automation, Helm for Kubernetes deployments, and Prometheus/CloudWatch-style monitoring.

One important interview rule: use the following as realistic scenarios to understand and prepare for. Only describe an incident as something you personally faced if it actually happened; otherwise, say how you would troubleshoot it.

For every responsibility, we'll cover:

- Technical definition in simple language.
- A natural project story to remember the workflow.
- A realistic production task and incident.
- Common errors and troubleshooting.

---

1. Infrastructure provisioning and configuration management using Ansible

Core concept: Automation

1. Simple technical definition

Ansible is an automation and configuration management tool used to configure multiple servers remotely without manually logging into each server.

We write instructions in YAML called *playbooks*. Ansible connects to Linux servers through SSH and performs tasks such as:

- Installing packages.
- Creating users and groups.
- Configuring directories and permissions.
- Managing system services.
- Installing monitoring agents.
- Applying server configurations.

Remember: Ansible helps make multiple servers consistent and reduces manual configuration effort.

2. Storytelling – Project explanation

" In our project, we had multiple Linux servers supporting the application across different environments. Initially, configuring servers manually was time-consuming, especially when we had to prepare new environments or replace existing instances.

We used Ansible playbooks to automate common server configurations. For example, whenever a new server was provisioned, we could execute a playbook to install the required packages, configure users, set permissions, update system configurations and start the necessary services.

This helped us maintain consistency across servers and reduced manual intervention during deployments."

3. Real-world task with an incident

Task: Preparing a new application server.

Imagine we launch three new EC2 instances for an application environment.

Workflow:

1. Provision EC2 instances using AWS or Terraform.
2. Add their private IP addresses to the Ansible inventory.
3. Execute an Ansible playbook.
4. Install Java, required packages and dependencies.
5. Create application users and directories.
6. Configure permissions.
7. Start required services.
8. Validate whether all servers are configured properly.

Example inventory:

```ini
[app_servers]
10.0.1.10
10.0.1.11
10.0.1.12
```

Example playbook:

```yaml
- name: Configure application servers
  hosts: app_servers
  become: yes
```

```yaml
  tasks:
    - name: Install Java
      package:
        name: java-17-openjdk
        state: present

    - name: Create application user
      user:
        name: appuser
        state: present

    - name: Create application directory
      file:
        path: /opt/myapp
        state: directory
        owner: appuser
        mode: '0755'
```

Incident: Application deployment failed because Java was missing on one server.

Instead of manually logging into every server, we would:

- Check which server missed the dependency.
- Verify Ansible inventory and connectivity.
- Re-run the playbook.
- Validate Java installation.
- Retry application deployment.

4. Common errors and issues

Issue	Possible reason	Troubleshooting
SSH connection failed	Wrong key or security group	Check SSH access and credentials
Permission denied	Missing sudo privileges	Use `become: yes`
Package not found	Repository issue	Verify package repositories
Playbook failed	Incorrect YAML indentation	Validate YAML syntax
Service won't start	Configuration/dependency issue	Check `systemctl` and logs
Host unreachable	Incorrect inventory IP	Verify inventory and connectivity

Useful commands:

```bash
</> Bash
```

```
ansible all -m ping
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml --check
ansible-playbook site.yml
```

Memory connection: New server → Ansible connects → installs software → configures users → sets permissions → starts services → validates readiness.

---

2. Reusable Ansible playbooks and roles for Kubernetes cluster configuration and patching

Core concept: Reusability + Standardization

1. Simple technical definition

Ansible roles are a structured way of organizing automation tasks into reusable components.

Instead of writing one large playbook containing everything, we divide it into smaller roles.

For example:

- `docker_setup` : Container runtime configuration.
- `kubernetes_setup` : Kubernetes package installation.
- `monitoring_setup` : Monitoring agent installation.
- `security_setup` : OS-level security configurations.
- `patching` : Operating system package updates.

The same roles can be reused across development, QA, UAT and production environments.

2. Storytelling – Project explanation

"As our infrastructure grew, maintaining separate playbooks for every environment became difficult. Some configurations were repeated across development, QA and production.

To improve maintainability, we structured our Ansible automation using reusable roles. Each role handled a particular responsibility, such as Kubernetes node prerequisites, package installation, monitoring agent configuration and OS patching.

This allowed us to reuse the same automation across environments while maintaining environment-specific variables. It also helped us reduce configuration differences between environments."

3. Real-world task with an incident

Task: Applying security patches to Kubernetes worker nodes.

Imagine a security update is released for packages installed on our worker nodes.

A controlled workflow would be:

1. Identify affected nodes and required patches.
2. Validate the patch in development.
3. Drain one worker node at a time.
4. Apply package updates through Ansible.
5. Reboot if required.
6. Validate kubelet and container runtime.
7. Uncordon the node.
8. Verify workloads are healthy.
9. Repeat for remaining nodes.

Useful Kubernetes commands:

```bash
kubectl drain worker-1 --ignore-daemonsets
kubectl get nodes
kubectl uncordon worker-1
```

Ansible role structure:

```plaintext
roles/
  kubernetes/
    tasks/
      main.yml
    handlers/
      main.yml
    templates/
    defaults/
    vars/
```

Example:

```yaml
- name: Install Kubernetes packages
  hosts: workers
  become: yes
```

```
  roles:
    - kubernetes
    - monitoring
```

Incident: A worker node doesn't rejoin the cluster after patching.

Troubleshooting:

- Check node status.
- Verify kubelet service.
- Check container runtime.
- Validate node-to-control-plane connectivity.
- Review kubelet logs.
- Check whether the node has compatible Kubernetes packages.

```
</> Bash

systemctl status kubelet
journalctl -u kubelet -xe
kubectl get nodes -o wide
```

4. Common errors and issues

Issue	Possible reason	Troubleshooting
Role not found	Incorrect role path	Verify role directory
Variable conflict	Incorrect variable precedence	Review defaults and vars
Package version conflict	Incompatible versions	Pin supported versions
Node NotReady	Kubelet/runtime failure	Check service and logs
Pods disrupted during patching	Insufficient capacity or unsafe drain	Check PDBs and capacity
Patch failed	Repository/network issue	Verify package repository

Important client-level point: Never blindly patch every production worker node simultaneously. Use a controlled rolling maintenance approach, preserve workload availability, and verify node health before proceeding.

Memory connection: Reusable roles → Environment-specific variables → Controlled patching → Drain → Update → Validate → Uncordon.

---

3. End-to-end Jenkins pipeline for frontend compilation, containerization and Kubernetes deployment

Core concept: CI/CD Automation

1. Simple technical definition

Jenkins is an automation server used to implement Continuous Integration and Continuous Delivery/Deployment.

A Jenkins pipeline automates the journey from source code to application deployment.

For a frontend application, the typical workflow is:

1. Developer pushes code to Git.
2. Jenkins detects the change.
3. Dependencies are installed.
4. Frontend application is compiled.
5. Build artifacts are generated.
6. Docker image is created.
7. Image is pushed to Amazon ECR.
8. Kubernetes deployment is triggered.
9. Deployment health is validated.

2. Storytelling – Project explanation

"In our project, developers regularly committed frontend changes to Git. Earlier, building and deploying the application involved several manual steps, which increased deployment time and the possibility of human errors.

We automated this process using Jenkins pipelines. Whenever code changes were pushed, Jenkins checked out the source code, installed dependencies and executed the frontend production build.

After successful compilation, it created a Docker image, tagged it with the build version and pushed it to Amazon ECR. The deployment stage then updated the image reference in our Kubernetes deployment through Helm or Kubernetes manifests.

Finally, we validated the rollout to ensure that the application pods were running successfully."

3. Real-world task with an incident

Task: Deploying a new frontend release.

Jenkins CI/CD workflow

1. **Git checkout**Pull source code from GitHub/GitLab.

2. **Dependency installation** `npm install` or `npm ci`
3. **Frontend build** `npm run build`
4. **Docker image creation** `docker build`
5. **Push to ECR**Authenticate and push tagged image.
6. **Kubernetes deployment**Update image using Helm or manifests.
7. **Validation**Check rollout, pods and application health.

Example Jenkinsfile:

```groovy
pipeline {
    agent any

    environment {
        IMAGE = "myapp:${BUILD_NUMBER}"
    }

    stages {
        stage('Checkout') {
            steps {
                git 'https://github.com/org/frontend.git'
            }
        }

        stage('Build') {
            steps {
                sh 'npm ci'
                sh 'npm run build'
            }
        }

        stage('Docker Build') {
            steps {
                sh 'docker build -t $IMAGE .'
            }
        }

        stage('Push Image') {
            steps {
                echo 'Authenticate and push to ECR'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploy using Helm'
```

```
            }
        }
    }
}
```

This is a simplified illustration; production pipelines also require credential management, image tagging, ECR authentication and proper deployment commands.

Incident: Jenkins pipeline fails during frontend compilation.

Possible reasons:

- Dependency version mismatch.
- Incorrect Node.js version.
- Missing environment variables.
- TypeScript compilation errors.
- Insufficient build agent resources.

Troubleshooting:

```bash
node --version
npm --version
npm ci
npm run build
```

If the build succeeds locally but fails in Jenkins, compare the Node.js version, environment variables and dependency lockfile.

4. Common errors and issues

Issue	Possible reason	Troubleshooting
Git checkout failure	Credentials or branch issue	Check SCM credentials
npm install failure	Dependency conflict	Check package-lock.json
Build failure	Compilation errors	Inspect console output
Docker build failure	Incorrect Dockerfile	Validate build context
ECR push denied	IAM permission issue	Check ECR permissions
ImagePullBackOff	Wrong image/tag or registry access	Check pod events and image
Deployment timeout	Pods not becoming ready	Check rollout and pod logs

Memory connection: Code → Build → Image → Registry → Deployment → Validation.

4. Created and managed Helm charts and Kubernetes manifests

Core concept: Deployment standardization

1. Simple technical definition

Kubernetes manifests are YAML configuration files used to define Kubernetes resources such as:

- Deployment
- Service
- ConfigMap
- Secret
- Ingress
- HPA

Helm is a Kubernetes package manager. It allows us to organize multiple Kubernetes manifests into reusable charts and deploy applications with environment-specific configurations.

Simple difference:

- Kubernetes manifest = Defines what should run.
- Helm chart = Packages those definitions and makes them reusable and configurable.

2. Storytelling – Project explanation

"In our project, we had different Kubernetes environments such as development, QA, UAT and production. The application deployment configuration was mostly similar, but certain values like replica counts, resource limits, image tags and environment variables differed.

Managing separate YAML files for every environment became difficult.

So we used Helm charts to standardize the deployment process. We maintained common templates for deployments, services and ingress configurations, while environment-specific values were maintained in separate values files.

Whenever a new application version was released, we updated the image tag and performed a Helm upgrade. This made deployments more consistent and easier to manage."

3. Real-world task with an incident

Task: Deploying the same application into QA and production.

Example Helm structure:

```
myapp/
  Chart.yaml
  values.yaml
  templates/
    deployment.yaml
    service.yaml
    ingress.yaml
```

Environment-specific files:

```
values-dev.yaml
values-qa.yaml
values-prod.yaml
```

Example values:

```yaml
replicaCount: 3

image:
  repository: 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp
  tag: "v1.2.0"

resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

Deployment commands:

```bash
helm lint ./myapp

helm template myapp ./myapp \
```

```
  -f values-qa.yaml

helm upgrade --install myapp ./myapp \
  -f values-qa.yaml \
  -n qa
```

Incident: Deployment succeeded in QA but failed in production.

Possible reason: Production values contained an incorrect image tag or resource configuration.

Troubleshooting:

1. Check Helm release status.
2. Compare QA and production values.
3. Check Kubernetes events.
4. Validate the rendered manifest.
5. Correct the configuration.
6. Perform another Helm upgrade.

```
</> Bash

helm list -n prod
helm status myapp -n prod
helm get values myapp -n prod
kubectl get pods -n prod
kubectl describe pod <pod-name> -n prod
```

4. Common errors and issues

Issue	Possible reason	Troubleshooting
Helm upgrade failed	Invalid template	Run `helm lint`
ImagePullBackOff	Incorrect image tag	Verify ECR image
Release stuck	Pods not becoming ready	Check pod events
Wrong environment config	Incorrect values file	Verify values
Resource already exists	Resource ownership conflict	Check existing resources
Service unavailable	Incorrect selectors	Compare Service and Pod labels

Memory connection: Common templates → Different values → Helm release → Upgrade → Validate.

---

5. Kubernetes resource management – CPU, Memory, ResourceQuota and LimitRange

Core concept: Resource optimization

1. Simple technical definition

Kubernetes resource management controls how much CPU and memory an application container can request and consume.

There are four important concepts:

Concept	Meaning
Requests	Minimum resources Kubernetes uses for scheduling
Limits	Maximum CPU/memory a container can consume
ResourceQuota	Maximum resource consumption allowed within a namespace
LimitRange	Defines default and minimum/maximum resource settings for containers

Remember:

- Requests = Scheduling requirement.
- Limits = Consumption boundary.
- Quota = Namespace boundary.
- LimitRange = Container-level defaults and boundaries.

2. Storytelling – Project explanation

"In our application, multiple microservices were running on the same Kubernetes cluster. Some services consumed more CPU and memory than expected, which occasionally affected other workloads.

To prevent this, we defined CPU and memory requests and limits for our application containers.

We also used ResourceQuota to control the total resource consumption within namespaces and LimitRange to enforce default and maximum resource values.

This helped Kubernetes schedule pods more effectively and prevented one application from consuming excessive cluster resources."

3. Real-world task with an incident

Task: Preventing a backend service from consuming excessive memory.

Imagine a backend API starts processing a large number of requests and its memory consumption increases.

Example deployment:

```yaml
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"

  limits:
    cpu: "1"
    memory: "1Gi"
```

Here:

- `500m` means half a CPU core.
- `1` means one CPU core.
- `512Mi` means 512 MiB memory.
- `1Gi` means 1 GiB memory.

ResourceQuota example:

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: app-quota
  namespace: production
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"
```

Incident: Pods are getting OOMKilled during traffic spikes.

OOMKilled means the container exceeded its available memory limit and was terminated.

Troubleshooting:

```bash
kubectl get pods -n production
```

```
kubectl describe pod <pod-name> -n production

kubectl top pods -n production

kubectl top nodes
```

We would:

1. Identify the affected pod.
2. Check memory consumption.
3. Inspect application logs.
4. Determine whether it is a memory leak or insufficient allocation.
5. Tune requests and limits based on actual usage.
6. Monitor the application after the change.

4. Common errors and issues

Issue	Reason	Troubleshooting
OOMKilled	Memory limit exceeded	Analyze memory usage
Pending pods	Insufficient node resources	Check requests and node capacity
CPU throttling	CPU limit too restrictive	Review CPU usage and limits
Quota exceeded	Namespace quota reached	Check ResourceQuota
Pod rejected	LimitRange violation	Check default/min/max values
Node pressure	Excessive resource consumption	Check node metrics and workloads

Important interview point: CPU limits can cause throttling, whereas exceeding a container's memory limit can result in OOMKilled.

Memory connection: Requests for scheduling → Limits for consumption → Quota for namespace → LimitRange for container rules.

---

6. Application autoscaling using HPA

Core concept: Dynamic scaling

1. Simple technical definition

HPA stands for Horizontal Pod Autoscaler.

It automatically increases or decreases the number of application pods based on resource utilization or other configured metrics.

For example:

- Normal traffic: 3 pods.
- Increased traffic: 6 pods.
- Heavy traffic: 10 pods.
- Traffic decreases: Scale back to 3 pods.

HPA commonly uses CPU and memory utilization metrics.

It requires a metrics source, commonly Metrics Server for resource-based metrics.

2. Storytelling – Project explanation

"Our application was accessed by users across different regions of India, and traffic was not consistent throughout the day.

During peak business hours, the number of incoming requests increased significantly. Running a fixed number of pods was not sufficient to handle these variations efficiently.

To address this, we configured Kubernetes HPA for important application deployments.

We defined minimum and maximum replica counts and configured CPU and memory utilization targets. When resource utilization increased, HPA automatically increased the pod count. When traffic reduced, it gradually scaled down the replicas.

This helped us maintain application performance while using cluster resources more efficiently."

3. Real-world task with an incident

Task: Scaling backend pods during peak traffic.

Example HPA:

```yaml
</> YAML

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

metadata:
  name: backend-hpa

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
```

```
  minReplicas: 3
  maxReplicas: 10

  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70

    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
```

Useful commands:

```
</> Bash

kubectl get hpa
kubectl describe hpa backend-hpa
kubectl top pods
kubectl get deployment backend
```

Incident: Traffic increases, but HPA does not scale pods.

Possible causes:

- Metrics Server is unavailable.
- CPU or memory requests are missing.
- Metrics are not being collected.
- HPA configuration is incorrect.
- Maximum replica count has already been reached.

Troubleshooting:

```
</> Bash

kubectl top nodes
kubectl top pods
kubectl describe hpa backend-hpa
kubectl get apiservices
```

4. Common errors and issues

Issue	Possible reason	Troubleshooting
HPA shows Unknown	Metrics unavailable	Check Metrics Server
Pods not scaling	Missing resource requests	Define requests
Scaling reaches maximum	MaxReplicas reached	Review capacity
New pods remain Pending	Insufficient node capacity	Check node resources
Frequent scale up/down	Fluctuating metrics	Tune scaling behavior
Traffic remains high	CPU is not the bottleneck	Analyze application metrics

Important distinction for interviews:

- HPA = Adds or removes pods.
- Cluster Autoscaler = Adds or removes worker nodes.
- Karpenter = Provisions and consolidates nodes based on workload requirements.

HPA alone cannot create additional EC2 capacity if the cluster has no room for new pods.

Memory connection: Traffic increases → CPU/memory increases → HPA detects → Pods increase → Traffic decreases → Pods reduce.

---

7. Rolling updates and rollback strategies

Core concept: Zero/minimal downtime deployment

1. Simple technical definition

Rolling updates allow Kubernetes to gradually replace old application pods with new application pods without terminating all existing pods simultaneously.

Rollback means returning to a previously working application version when a new deployment causes problems.

For example:

- Current version: v1.
- New version: v2.
- Kubernetes gradually replaces v1 pods with v2.
- If v2 fails, we can roll back to v1.

2. Storytelling – Project explanation

"In our project, application updates were released regularly, so deploying a new version without affecting existing users was important.

We used Kubernetes Deployment rolling update strategies. Instead of stopping all existing pods, Kubernetes gradually created new pods and terminated old ones based on the configured rollout strategy.

We also configured readiness probes to ensure new pods were ready to receive traffic before they were considered available.

If a new deployment introduced issues, we could use Kubernetes rollout history and rollback commands to return to the previous stable version.

This helped us reduce deployment risks and maintain application availability."

3. Real-world task with an incident

Task: Releasing a new backend version.

Example Deployment configuration:

```yaml
</> YAML

spec:
  replicas: 4

  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

Meaning:

- Four replicas are expected.
- Kubernetes can temporarily create one additional pod.
- It should not intentionally make any of the desired four replicas unavailable during the rollout.

Useful commands:

```bash
</> Bash

kubectl rollout status deployment/backend

kubectl rollout history deployment/backend
```

```
kubectl rollout undo deployment/backend

kubectl rollout restart deployment/backend
```

Incident: New application version is deployed, but users start receiving HTTP 500 errors.

Possible reasons:

- Application bug.
- Incorrect environment variables.
- Database connectivity failure.
- Incompatible API changes.
- Incorrect application configuration.

Troubleshooting:

1. Check deployment rollout status.
2. Inspect the new pod logs.
3. Check readiness and liveness probes.
4. Verify application dependencies.
5. If the release is faulty, roll back.
6. Validate application health.

```
</> Bash
kubectl get pods

kubectl logs <pod-name>

kubectl describe pod <pod-name>

kubectl rollout undo deployment/backend
```

4. Common errors and issues

Issue	Possible reason	Troubleshooting
Rollout stuck	New pods not ready	Check readiness probes
ImagePullBackOff	Invalid image	Verify registry and tag
CrashLoopBackOff	Application startup failure	Inspect logs
Deployment unavailable	Insufficient capacity	Check resources
Rollback unsuccessful	Revision unavailable or deployment configuration issue	Inspect rollout history

User-facing downtime	Insufficient replicas or bad traffic handling	Check availability and readiness

Important interview point: Rolling updates help achieve zero or minimal downtime, but they do not guarantee it automatically. Readiness probes, sufficient capacity, application compatibility, graceful shutdown and appropriate deployment settings are also necessary.

For stronger production protection, we can use PodDisruptionBudgets, though PDBs primarily govern voluntary disruptions and do not replace deployment rollout configuration.

Memory connection: New version → Create new pods → Readiness check → Replace old pods → Validate → Rollback if faulty.

---

8. Container networking – DNS, Service routing, Ingress and Route controllers

Core concept: Application connectivity

1. Simple technical definition

Kubernetes networking allows pods and services to communicate internally and externally.

There are four important areas:

Component	Purpose
CoreDNS	Resolves Kubernetes service DNS names
Service	Provides stable network access to pods
Ingress	Routes external HTTP/HTTPS traffic into the cluster
Ingress Controller	Implements and manages Ingress routing rules

In AWS EKS, AWS Load Balancer Controller is commonly used to provision and manage AWS Application Load Balancers for Kubernetes Ingress resources.

Important: OpenShift uses Route resources as an alternative to standard Kubernetes Ingress for exposing applications.

2. Storytelling – Project explanation

"In our project, different microservices were deployed as Kubernetes pods, and they needed to communicate with each other internally.

We used Kubernetes Services to provide stable endpoints instead of directly depending on changing pod IP addresses.

CoreDNS handled service name resolution, allowing one microservice to communicate with another using Kubernetes DNS names.

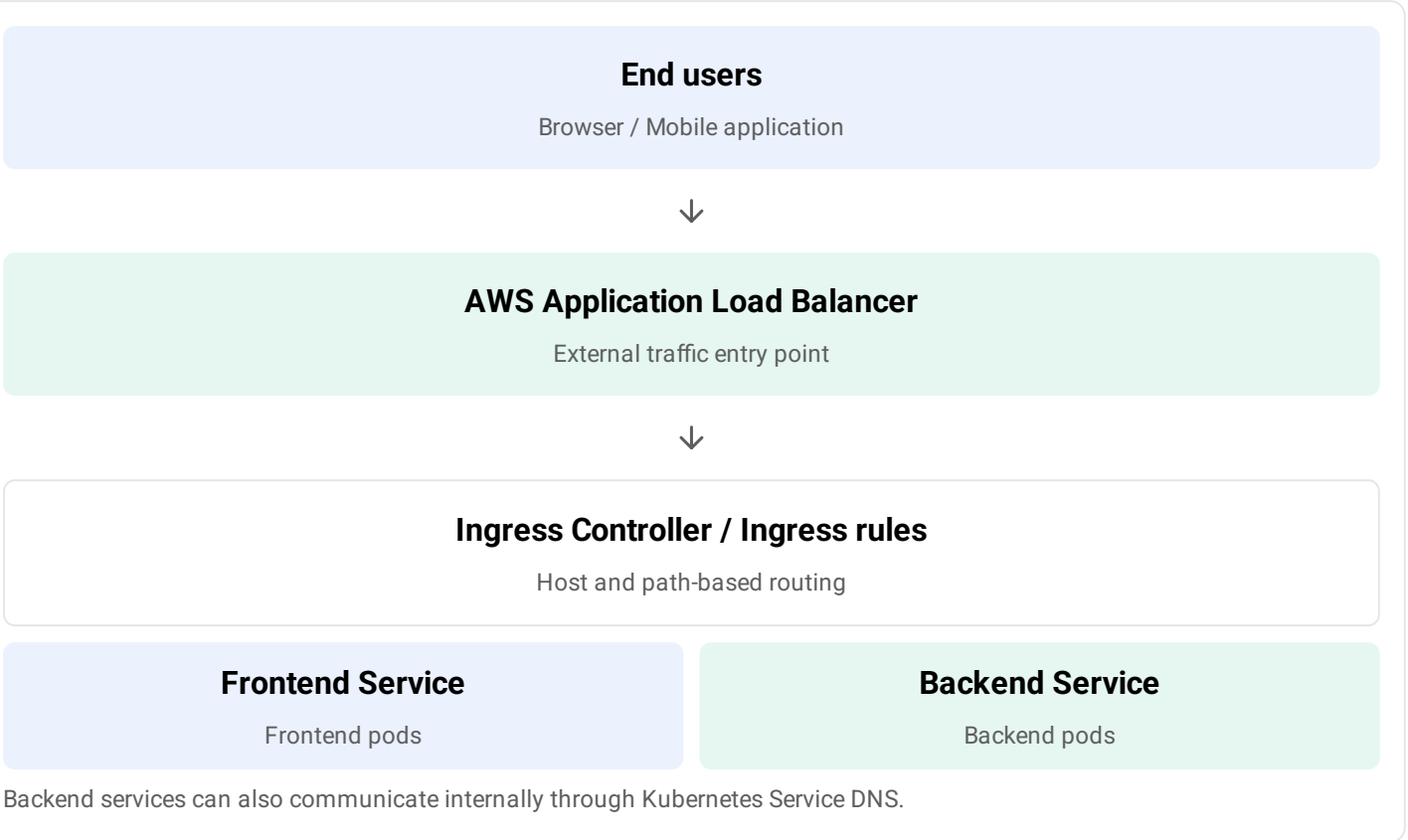
For external traffic, we configured Ingress rules and an appropriate Ingress Controller to route incoming requests to the correct application services.

We also monitored DNS resolution, service connectivity and ingress routing whenever application connectivity issues occurred."

3. Real-world task with an incident

Task: Exposing a frontend and backend application.

Imagine the architecture:



Example Kubernetes Service:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-service

spec:
  selector:
    app: backend
```

```
  ports:
    - port: 80
      targetPort: 8080

  type: ClusterIP
```

Example DNS name:

```
</> Plain text

backend-service.production.svc.cluster.local
```

Example Ingress:

```
</> YAML

apiVersion: networking.k8s.io/v1
kind: Ingress

metadata:
  name: app-ingress

spec:
  ingressClassName: alb

  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

This example assumes AWS Load Balancer Controller is installed and configured.

Incident: Frontend application is unable to communicate with the backend service.

Possible causes:

- CoreDNS failure.

- Incorrect Service name.
- Incorrect Service selector.
- Incorrect target port.
- NetworkPolicy blocking traffic.
- Application not listening on the expected port.

Troubleshooting:

```bash
kubectl get pods -n production

kubectl get svc -n production

kubectl get endpoints backend-service

kubectl get pods -n kube-system

kubectl logs -n kube-system \
  deployment/coredns
```

From inside a test pod:

```bash
nslookup backend-service.production.svc.cluster.local

curl http://backend-service.production.svc.cluster.local
```

4. Common errors and issues

Issue	Possible reason	Troubleshooting
DNS resolution failed	CoreDNS issue	Check CoreDNS pods/logs
Service unavailable	Incorrect selectors	Check endpoints
Connection refused	Wrong target port	Verify application port
502 Bad Gateway	Backend unavailable	Check service and pod health
503 Service Unavailable	No healthy backend endpoints	Inspect endpoints/readiness
Ingress not working	Controller or ingress class issue	Check controller logs
External traffic blocked	Security group/NACL issue	Validate AWS networking

Memory connection: User request → Load Balancer → Ingress → Service → Pod. Internal communication → Service DNS → CoreDNS → Destination Service.

Final revision – Connect all eight responsibilities into one project story

This is the bigger picture you should be able to explain when the interviewer asks:

"Can you explain your DevOps responsibilities in your previous project?"

Your end-to-end DevOps workflow

1. **Infrastructure and server configuration – Ansible**Prepare Linux servers, install dependencies and maintain consistent configurations.
2. **Reusable automation – Ansible Roles**Standardize configurations and automate repeatable maintenance tasks.
3. **CI/CD – Jenkins**Checkout code, compile frontend, build Docker image and push to ECR.
4. **Application packaging – Helm**Maintain reusable Kubernetes deployment templates and environment-specific configurations.
5. **Resource management – Kubernetes**Define CPU/memory requests, limits, quotas and container defaults.
6. **Autoscaling – HPA**Automatically scale application pods according to resource utilization.
7. **Deployment strategy – Rolling Updates**Release new application versions safely and roll back when required.
8. **Networking – CoreDNS, Services and Ingress**Maintain internal communication and external application accessibility.

Client interview – Questions you should be ready for

These are the cross-questions I would prioritize. They test whether you understand the actual engineering work rather than just tool definitions.

Priority	Interview question
Very high	How does Ansible connect to remote servers?
Very high	Difference between Ansible playbook and role?
Very high	How do you patch Kubernetes worker nodes?
Very high	Explain your Jenkins pipeline from Git to EKS.
Very high	How do you push Docker images to ECR?
Very high	Difference between Helm and Kubernetes YAML?
Very high	What happens when a pod gets OOMKilled?

Very high	How does HPA calculate pod scaling?
Very high	Difference between HPA and Cluster Autoscaler?
Very high	How do you perform a zero-downtime deployment?
High	How do you roll back a failed deployment?
High	How does CoreDNS work?
High	Difference between ClusterIP, NodePort and LoadBalancer?
High	What happens when a Service has no endpoints?
High	How does Ingress work in EKS?

My recommendation: Prepare in this order

Given that you have theoretical knowledge but limited production exposure, don't try to memorize all eight stories at once.

1. Jenkins CI/CD – understand the entire pipeline.
2. Kubernetes deployment and Helm – understand how the application reaches EKS.
3. Kubernetes resource management and HPA – understand application performance.
4. Kubernetes networking – understand connectivity and troubleshooting.
5. Ansible and patching – understand infrastructure operations.

One final interview tip: For every technology, learn three things especially well: what you configured, why you configured it, and what you would check when it fails. Those three things will make your answers much more convincing than memorizing lengthy project stories.